

PRÁCTICA 3

Sincronización de procesos con mutex: Carreras críticas en el problema del productor-consumidor

ÍNDICE

Introducción	1
Códigos	1
Conclusiones	2

INTRODUCCIÓN

Para el apartado 1 de la práctica, se pide implementar el código del problema del productor-consumidor visto en las clases teóricas, con el objetivo de presenciar las carreras críticas producidas, además de la evolución de ambos programas.

Las carreras críticas buscadas serán forzadas mediante `sleeps`, que obligarán a que ocurran cambios de contexto mientras los procesos se sitúan en sus regiones críticas.

CÓDIGOS

Se trabajará con 1 único programa, que es el siguiente:

`prod_cons_thread.c` - Código que implementa el problema del productor-consumidor utilizando hilos.

COMPILACIÓN - `gcc -o prod_cons_thread prod_cons_thread.c -pthread`

EJECUCIÓN - `./prod_cons_thread`

Es necesario que en el mismo directorio del ejecutable exista un archivo de texto con el nombre `archivo.txt`, el cual contenga números enteros separados por espacios.

FUNCIONAMIENTO - Tras `inicializar` las `variables` pertinentes, poniendo los elementos del `buffer` a 0 y las sumas también, se `crean dos hilos`, uno para el productor y otro para el consumidor, y el `thread main` esperará por ambos mediante un `join`.

En el **productor** se hace lo siguiente,

1. Se `abre` el archivo `archivo.txt` en modo de lectura.
2. Se entra a un `bucle` de 80 iteraciones, que también puede terminar en caso de que se llegue al final del archivo leído.
3. Se ejecuta `produce_item`, con lo que se lee el siguiente entero del archivo y se añade a la suma del productor. En caso de obtener un EOF, se considera el entero 101 y no se suma.

4. Se produce una **espera activa** mientras el *buffer* está lleno (se recorre la cola hasta encontrar un elemento igual a 0).
5. [Opcional] Se **duerme** unos segundos para aumentar la probabilidad de condiciones de carrera.
6. Se ejecuta **insert_item**, con lo que se busca la siguiente posición vacía del *buffer* para escribir ahí el entero leído y se actualiza **siguiente_posicion** (esta variable se emplea para dotar al *buffer* del comportamiento FIFO pedido).
7. Fuera del bucle, en caso de completar las 80 iteraciones y no alcanzar el EOF del archivo, se **inserta un código 101 en el buffer** y se **cierra archivo.txt**.

Simultáneamente, en el **consumidor** se hace lo siguiente:

1. Se entra a un **bucle** de 80 iteraciones, que también puede terminar en caso de que se haya leído un código 101 de EOF y el *buffer* esté vaciado por completo.
2. Se produce una **espera activa** mientras el bucle está vacío (se recorre la cola hasta encontrar un elemento distinto de 0).
3. [Opcional] Se **duerme** una cierta cantidad de segundos para incrementar la probabilidad de condiciones de carrera.
4. Se ejecuta **remove_item**, con lo que se retira el elemento del *buffer* más antiguo en ser insertado (gracias al entero auxiliar **siguiente_posicion**) y se sustituye por un 0. También se comprueba si se ha leído un 101 y si se ha vaciado la cola.
5. Se ejecuta **consume_item**, con lo que se añade el entero leído a la suma del consumidor.

Finalmente, desde el hilo **main**, se **imprimen las sumas** de enteros calculada por el productor y por el consumidor, y se muestra la **diferencia entre ambas**.

CONCLUSIONES

Al ejecutar el programa con un archivo de ejemplo, con el contenido "58 19 61 36 26 52 68 45 21 5 96 52 27 15 18 3 2 1 54", se observan los siguientes resultados:

- Si ninguno de los hilos duerme, las **sumas** del consumidor y del productor **valen lo mismo** (659). Aun así, es destacable que el consumidor retira el *item* 0 en ciertos momentos, lo que permite concluir que **sí se producen carreras críticas**.
- Si el productor duerme y el consumidor no lo hace, el **proceso nunca termina**. Esto se debe a que el consumidor retira el *item* 0 numerosas veces del *buffer*, antes de que su *quantum* termine; de este modo, este hilo completa sus 80 iteraciones máximas antes de que el productor recorra todo el archivo.
- Si el consumidor duerme y el productor no lo hace, las **sumas** de ambos hilos **coinciden**. Esto sucede porque la velocidad reducida del consumidor causa que el productor llene de inmediato el

buffer y deba esperar por algún hueco. Es decir, la lentitud del consumidor ralentiza la ejecución del productor.

- Si ambos hilos duermen, las sumas del productor y del consumidor valen lo mismo. Además, a diferencia del caso sin sleeps, solo se consumen los enteros insertados en la cola por el productor, sin ningún 0 intermedio. Es decir, no se producen condiciones de carrera.

```
[PROD] Producido el item 19
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 0
[CONS] Consumido el item 19
```

Ejemplo de condición de carrera en la que el consumidor retira 0 del buffer durante el periodo de tiempo transcurrido entre que el productor lee el siguiente elemento del archivo y lo inserta en la cola.

Esta imagen se corresponde con el caso en el que el productor duerme y el consumidor no.